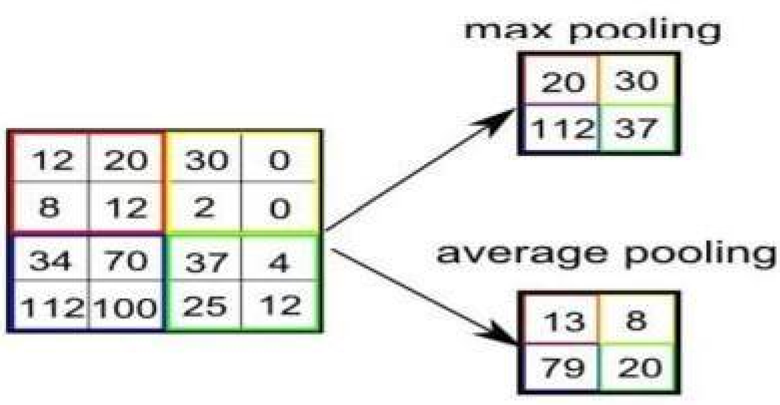
Aim: To understand the process of convolution over the image and apply over the classification problem

IDE: Google Colab

Theory:

Convolutional Neural Networks (CNN) are complex feed forward neural networks. CNNs are used for image classification and recognition because of its high accuracy There are three types of layers in a convolutional neural network: i. Convolutional layer ii. Pooling layer iii. Fully connected layer Each of these layers has different parameters that can be optimized and performs a different task on the input data.

What is Pooling Layer?
Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to decrease the computational power required to process the data through dimensionality reduction. There are two types of Pooling i. Average Pooling. ii. Max Pooling Max Pooling returns the maximum value from the portion of the image covered by the Kernel. On the other hand, Average Pooling returns the average of all the values from the portion of the image covered by the Kernel. Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction. On the other hand, Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.



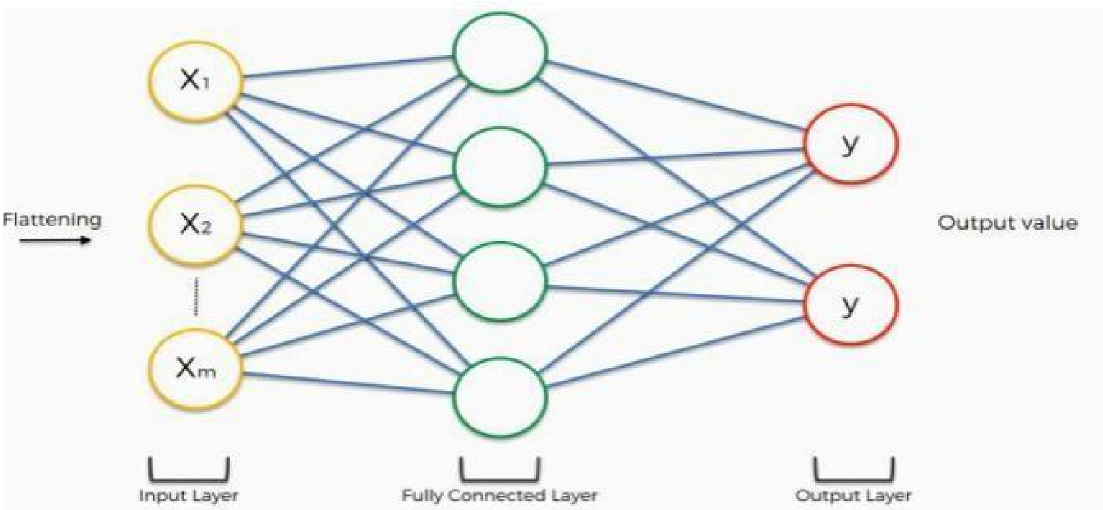
What is Convolutional Layer?
Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image. A convolutional layer contains a set of filters whose parameters need to be learned. The height and weight of the filters are smaller than those of the input volume. Each filter is convolved with the input volume to compute an activation map made of neurons.

What is Fully Connected Layer?
A fully connected layer that takes the output of convolution/pooling and predicts the best label to describe the image We have three layers in the full connection step i. Input layer ii. Fully-connected layer iii. Output layer

Input Layer: It takes the output of the previous layers, "flattens" them and turns them into a single vector that can be an input for the next stage.

Fully Connected Layer: It takes the inputs from the feature analysis and applies weights to predict the correct label.

Output Layer: It gives the final probabilities for each label.



ReLU Layer: ReLU is an activation function. Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable. The main aim is to remove all the negative values from the convolution. All the positive values remain the same but all the negative values get changed to zero.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Normalize the data
5. Pre-process the data
6. Visualize the Data
7. Write the CNN model function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Observe the cost function vs iterations learning curve

Program (Code):

```
# 1. Load the basic libraries and packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import keras
from keras.datasets import fashion_mnist
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split

# 2. Load the dataset
# Fashion MNIST dataset (images + labels)
(train_X,train_Y),(test_X,test_Y)=fashion_mnist.load_data()
```

```
# 3. Analyse the dataset
print(train_X.shape,train_Y.shape)
print(test_X.shape, test_Y.shape)
print("Unique classes:", np.unique(train_Y))
```

```
# 4. Normalize the data
# Convert pixel values from (0-255) to (0-1)
train_X = train_X.astype('float32') / 255
test_X = test_X.astype('float32') / 255
```

```
# 5. Pre-process the data
# Reshape data for CNN input (28x28x1)
train_X = train_X.reshape(-1,28,28,1)
test_X = test_X.reshape(-1,28,28,1)
```

```
# Convert labels into one-hot encoding
train_Y = to_categorical(train_Y)
test_Y = to_categorical(test_Y)
```

```
# Split into training and validation sets
train_X,valid_X,train_label,valid_label = train_test_split(
    train_X,train_Y,test_size=0.2,random_state=13
)
```

```
# 6. Visualize the Data
# Show one sample image
plt.imshow(train_X[0].reshape(28,28))
```

```
plt.title("Sample Image")
plt.show()
```

```
# 7. Write the CNN model function
model = tf.keras.Sequential()
```

```
# Convolution Layer 1
model.add(tf.keras.layers.Conv2D(32,(3,3),padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D((2,2)))
```

```
# Convolution Layer 2
model.add(tf.keras.layers.Conv2D(64,(3,3),padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D((2,2)))
```

```
# Convolution Layer 3
model.add(tf.keras.layers.Conv2D(128,(3,3),padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D((2,2)))
```

```
# Fully connected layers
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(128))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.Dense(10,activation='softmax'))
```

```
# 8. Write the Cost Function
# Using categorical crossentropy loss function
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

```
# 9. Gradient Descent Optimization Algorithm
# Adam optimizer is used (advanced form of gradient descent)
```

```
# 10. Apply training to minimize loss
history = model.fit(train_X,train_label,
                    batch_size=64,
                    epochs=20,
                    validation_data=(valid_X,valid_label))
```

```
# 11. Observe cost function vs iterations (learning curve)
loss = history.history['loss']
val_loss = history.history['val_loss']
```

```
epochs = range(len(loss))
```

```
plt.plot(epochs, loss, label='Training Loss')
plt.plot(epochs, val_loss, label='Validation Loss')
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Cost Function vs Iterations")
plt.legend()
plt.show()
```

Results:

To be attached with

a. Training dataset

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

b. Model summary

```
(60000, 28, 28) (60000,)
```

```
(10000, 28, 28) (10000,)
```

```
[0 1 2 3 4 5 6 7 8 9]
10
```

```
(60000, 28, 28, 1) (60000,)
(10000, 28, 28, 1) (10000,)
```

c. Training and validation accuracy w.r.t epochs before regularization

```
testing_evaluation=model.evaluate(test_X,test_Y_one_hot)
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - accuracy: 0.9190 - loss: 0.4888
```

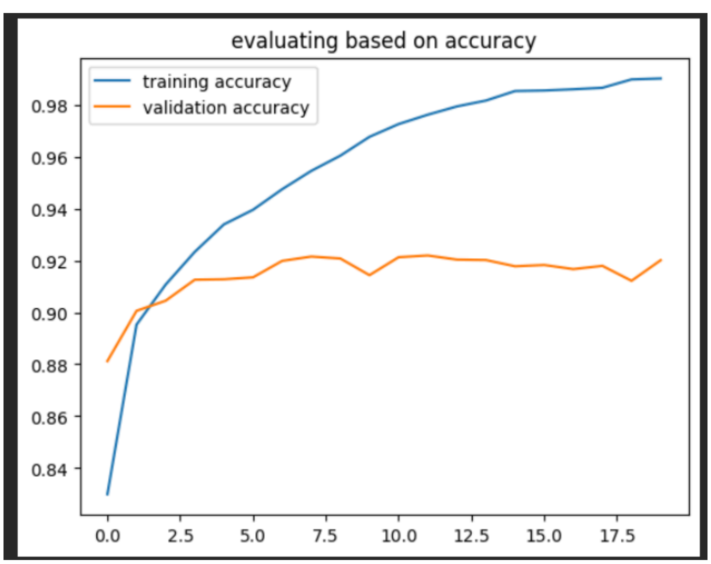
d. Training and validation loss w.r.t epochs before regularization

```
testing_evaluation  #loss,accuracy
[0.4649391174316406, 0.9186000227928162]
```

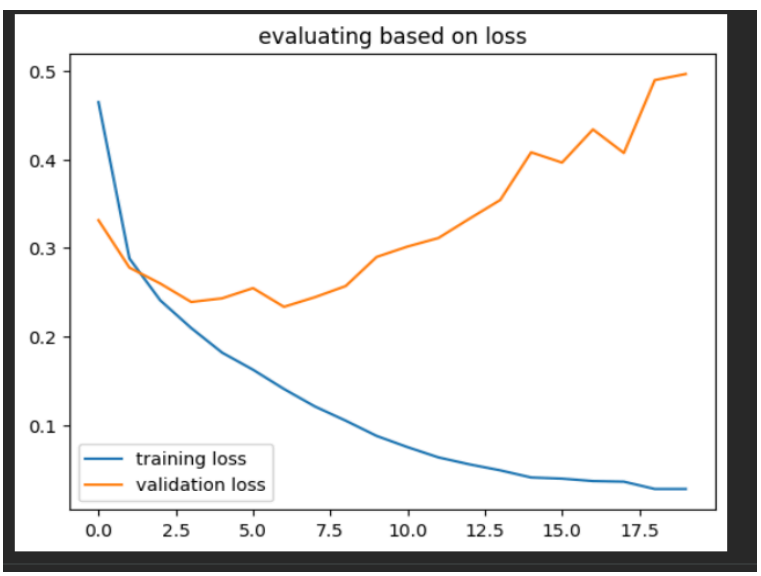
e. Training and validation accuracy w.r.t epochs after regularization

f.

g. Original v/s predicted labels for correct predicted observations



h. Original v/s predicted labels for incorrect predicted observations



Pre Lab Exercise:

a. What is Convolution process? Explain giving example.

Convolution is a process where a small filter (kernel) slides over an image to extract important features like edges or patterns. For example, in an image of a shirt, convolution helps detect edges, textures, or shapes by applying filters on pixel values.

b. What are different layers in CNN model?

A CNN model mainly consists of Convolutional layers (feature extraction), Activation layers (like ReLU), Pooling layers (dimension reduction), Fully Connected layers (classification), and Output layer (final prediction).

c. What is the requirement of the pooling layer in the CNN model?

Pooling reduces the size of feature maps, which helps in lowering computation and preventing overfitting. It also keeps important features while removing unnecessary details, making the model more efficient.

d. What is the requirement of the use of ReLU activation function after convolution step?

ReLU introduces non-linearity into the model, allowing it to learn complex patterns. Without ReLU, the model would behave like a linear model and fail to capture important features from images.

Observation and Result Analysis:

e. Nature of the dataset

The dataset used is Fashion MNIST, which consists of grayscale images of size 28×28 pixels representing different clothing items. It contains 10 classes and is well-structured for image classification tasks. The data is normalized and reshaped to make it suitable for CNN input.

f. Training Process without regularization

Without regularization, the CNN model learns quickly and training accuracy increases steadily. However, the validation accuracy may not improve at the same rate, indicating that the model starts overfitting and memorizing the training data.

g. After regularization in the training Process

After applying dropout layers for regularization, the model becomes more generalized. The training accuracy may slightly decrease, but validation accuracy improves, showing better performance on unseen data and reduced overfitting.

h. Observation over the Learning Curves

The learning curves show that training loss decreases continuously, while validation loss stabilizes after regularization. The gap between training and validation curves reduces, indicating improved model generalization and stability.

Post Lab Exercise:

a. **Why CNN is preferred over ANN for images**
CNN is preferred because it can automatically detect important features like edges and patterns using convolution filters. It also reduces the number of parameters compared to ANN, making it more efficient for image data.

b. **Can CNN be applied over Text data? If yes, then how. If no, then why?**
Yes, CNN can be applied to text data. Words are first converted into numerical vectors (embeddings), and then convolution is applied to capture patterns like phrases or important word combinations in the text.

c. **What is the role of dropout layer?**
Dropout randomly turns off some neurons during training, which helps prevent overfitting. It forces the model to learn more robust features instead of relying on specific neurons.

d. **What will happen if maxpooling is replaced with minpooling?**
If max pooling is replaced with min pooling, the model will capture the smallest values instead of the most important features. This may lead to loss of important information and reduce model performance.